# Individual Contribution Report

**Student Name:** Tamara Nicole Dondiego

**Group:** SureballNa (MiniOSNova)

**Assigned Role:** Memory Management Developer

### What did I contribute to the project?

- Developed the virtual memory management module (memory.py) to simulate system RAM allocation and deallocation in real time.
- Integrated commands to monitor system RAM status (memory), allocate memory chunks (allocate), and release memory back to the pool (free).
- Assisted in testing memory commands in the terminal and writing the Memory Management section of the User Manual.

### What features did I personally develop?

- **memory Command:** Displays the status of simulated system RAM, including Total Memory, Used Memory, and Free Memory.
- **allocate <MB> Command:** Reserves a specified amount of memory from the available pool and updates the system state.
- **free <MB> Command:** Releases a specified amount of previously allocated memory back to the available memory pool.

### What problems did I encounter?

- **Over-allocation Risk:** Attempting to allocate more memory than the available free space could lead to negative free memory values or program instability.
- **Invalid Deallocation:** Attempting to free more memory than was currently allocated, or entering negative values, resulted in inaccurate memory tracking.
- **Input Validation:** Non-numeric inputs for memory values, such as entering letters instead of megabytes, caused runtime crashes during testing.

### How did I solve those problems?

- Added a validation check during allocation (if amount > free_memory) to prevent over-allocation and display a “Not enough memory available” warning.
- Added boundary checks to the free command to ensure users cannot free more memory than is currently marked as used.
- Implemented try-except blocks around memory inputs to handle non-integer values gracefully with an error message instead of crashing.

### What did I learn from this project?

I learned how operating systems track and manage physical memory allocations dynamically, how to enforce memory constraints programmatically in Python, and how modular Python components communicate memory state changes to the main command-line interface.

### Evidence of my contribution:

- **Project Documentation:** Memory management command tests and contributions to Technical_Documentation.pdf and User_Manual.pdf in the docs/ folder.